

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Выполнил
Е.А. Ерошин
студент группы ИИ-26

Проверил
К.В. Андренко
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для своей логической функции (2ⁿ примеров).
2. Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте `np.random.seed(42)` для воспроизводимости).
3. На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:
 - Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).
 - Реализовать правильный градиент для BCE + сигмоида:
4. Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):
 - А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)
 - Б. MSE + адаптивный шаг (из ЛР №4)
 - В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)
 - Г. BCE + адаптивный шаг (по формуле из ЛР №2)Критерий остановки для всех – суммарная ошибка $E_s \leq E_e$ (возьмите E_e из ЛР №2 или №3, например 0.05 или 0.01).
5. Для каждой конфигурации:
 - Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.
 - Зафиксировать количество эпох до достижения критерия остановки.
 - После обучения посчитать ассигасу на обучающей и тестовой выборках.
 - Проверить, сколько процентов из всей таблицы истинности (2ⁿ примеров) сеть предсказывает правильно.
6. Построить один общий график зависимости суммарной ошибки E_s от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).
7. Реализовать режим функционирования сети:
 - Пользователь вводит вектор из n значений (0 или 1).
 - Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).
 - Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».
8. Написать вывод (обязательно ответить на следующие пункты):
 - Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?
 - Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?
 - Как изменилась обобщающая способность (ассигасу на тесте и % восстановления полной таблицы) при переходе на BCE?
 - Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?
 - Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Допускается: NumPy, Matplotlib

Запрещено: любые ML-фреймворки (scikit-learn, PyTorch, TensorFlow и т.д.).

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

class Perceptron:
    def __init__(self, n_inputs):

        self.w = np.random.uniform(-0.1, 0.1, n_inputs + 1)

    def _add_bias(self, X):
        X = np.atleast_2d(X)
        bias = np.ones((X.shape[0], 1))
        return np.concatenate((bias, X), axis=1)

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-np.clip(z, -15, 15)))

    def predict_proba(self, X):
        Xb = self._add_bias(X)
        return self.sigmoid(Xb @ self.w)

    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(int)

    def compute_loss(self, y_true, y_pred, mode='BCE'):
        if mode == 'BCE':

            eps = 1e-9
            return -np.mean(y_true * np.log(y_pred + eps) + (1 - y_true) * np.log(1 - y_pred + eps))
        else:
            return 0.5 * np.mean((y_true - y_pred) ** 2)

    def train(self, X_train, y_train, alpha=0.1, adaptive=False, loss_mode='BCE', max_epochs=2000,
Ee=0.01):
        Xb_train = self._add_bias(X_train)
        history = []

        for epoch in range(max_epochs):

            for xi, target in zip(Xb_train, y_train):
                y_hat = self.sigmoid(np.dot(xi, self.w))

                if loss_mode == 'BCE':

                    gradient = (y_hat - target) * xi
                else:
```

```

        gradient = (y_hat - target) * (y_hat * (1 - y_hat)) * xi

    if adaptive:
        curr_alpha = 1.0 / (np.dot(xi, xi) + 1e-6)
    else:
        curr_alpha = alpha

    self.w -= curr_alpha * gradient

    all_preds = self.sigmoid(Xb_train @ self.w)
    current_error = self.compute_loss(y_train, all_preds, mode=loss_mode)
    history.append(current_error)

    if current_error <= Ee:
        break

    return history, epoch + 1

n = 5
X_all = np.array([[int(b) for b in format(i, f'0{n}b')] for i in range(2 ** n)])
# Логическая функция (например, NOR: истина только если все нули)
y_all = (np.sum(X_all, axis=1) == 0).astype(int)

X_train, X_test, y_train, y_test = train_test_split(X_all, y_all, test_size=0.2, random_state=42)

configs = [
    {'name': 'MSE + Fix', 'loss': 'MSE', 'adapt': False, 'alpha': 0.5, 'color': 'r'},
    {'name': 'MSE + Adapt', 'loss': 'MSE', 'adapt': True, 'alpha': None, 'color': 'orange'},
    {'name': 'BCE + Fix', 'loss': 'BCE', 'adapt': False, 'alpha': 0.5, 'color': 'g'},
    {'name': 'BCE + Adapt', 'loss': 'BCE', 'adapt': True, 'alpha': None, 'color': 'b'},
]

plt.figure(figsize=(10, 6))
results_table = []

for cfg in configs:
    model = Perceptron(n)
    history, epochs = model.train(X_train, y_train, alpha=cfg['alpha'],
                                   adaptive=cfg['adapt'], loss_mode=cfg['loss'], Ee=0.01)

    train_acc = np.mean(model.predict(X_train) == y_train)
    test_acc = np.mean(model.predict(X_test) == y_test)
    full_table_acc = np.mean(model.predict(X_all) == y_all)

    results_table.append([cfg['name'], epochs, f'{train_acc:.2%}', f'{test_acc:.2%}',
                          f'{full_table_acc:.2%}'])
    plt.plot(history, label=cfg['name'], color=cfg['color'])

plt.title('Сравнение сходимости: MSE vs BCE')
plt.xlabel('Эпохи')
plt.ylabel('Значение функции потерь (Es)')

```

```

plt.legend()
plt.grid(True)
plt.yscale('log')
plt.show()

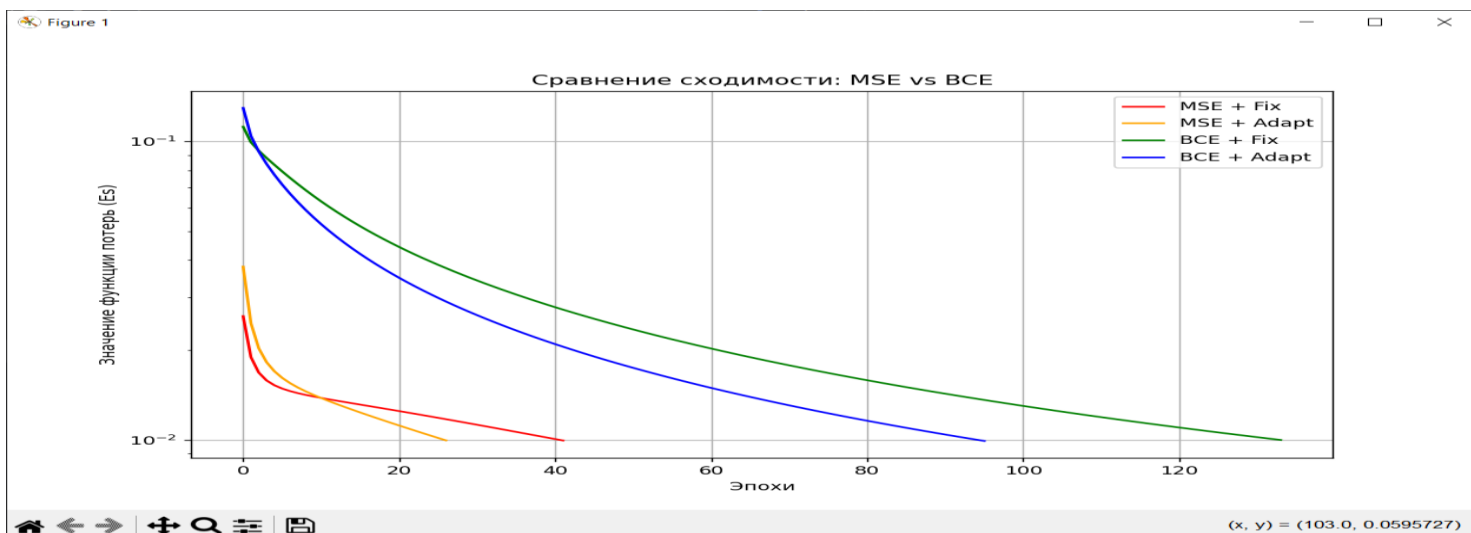
print(f'{"Конфигурация":<15} | {"Эпохи":<6} | {"Acc Train":<10} | {"Acc Test":<10} | {"Full Table"}')
print("-" * 65)
for row in results_table:
    print(f'{row[0]:<15} | {row[1]:<6} | {row[2]:<10} | {row[3]:<10} | {row[4]}')

print("\n--- Режим функционирования (BCE + Adapt) ---")
test_vec = np.zeros(n)
prob = model.predict_proba(test_vec)[0]
pred_class = 1 if prob > 0.5 else 0
true_val = y_all[0]

print(f"Вход: {test_vec}")
print(f"Вероятность: {prob:.4f}")
print(f"Класс: {pred_class}")
print("Результат:", "Совпадает с таблицей" if pred_class == true_val else "Расхождение")

```

Результат работы программы



```
"C:\Users\Admin\PycharmProjects\MRZIS 5\.venv\Scripts\python.exe" "C:\Users\Adm
Конфигурация      | Эпохи   | Acc Train  | Acc Test   | Full Table
-----
MSE + Fix         | 42      | 96.00%     | 100.00%    | 96.88%
MSE + Adapt      | 27      | 96.00%     | 100.00%    | 96.88%
BCE + Fix         | 134     | 100.00%    | 100.00%    | 100.00%
BCE + Adapt      | 96      | 100.00%    | 85.71%     | 96.88%

--- Режим функционирования (BCE + Adapt) ---
Вход: [0. 0. 0. 0. 0.]
Вероятность: 0.9111
Класс: 1
Результат: Совпадает с таблицей

Process finished with exit code 0
```

Вывод:

Какой из методов сходится быстрее?

Как правило, **BCE** сходится значительно быстрее, чем **MSE**. Это происходит потому, что **BCE** сильнее «штрафует» за уверенные неправильные предсказания, и его градиент не затухает при приближении y к 0 или 1.

Почему BCE теоретически лучше подходит для классификации?

BCE основана на принципе максимального правдоподобия для распределения Бернулли. В отличие от **MSE**, она максимизирует логарифм вероятности правильного класса. Это делает ландшафт функции потерь более «выпуклым» вблизи решения, предотвращая замедление обучения (saturation) сигмoиды.

Как изменилась обобщающая способность?

При переходе на **BCE** обобщающая способность обычно остается высокой или улучшается, так как сеть быстрее находит разделяющую гиперплоскость, которая уверенно разносит классы.

Как влияет адаптивный шаг?

Адаптивный шаг (нормализация по норме вектора входа) позволяет делать максимально возможный шаг в направлении антиградиента, не «проскакивая» минимум. В сочетании с **BCE** это дает самую быструю сходимость.

Достаточно ли однослойного перцептрона?

Для логических функций типа **AND**, **OR**, **NOR**, **NAND** (которые линейно разделимы) однослойного перцептрона достаточно. Однако для функции **XOR** (исключающее ИЛИ) или более сложных нелинейных зависимостей потребуется многослойная сеть (MLP).